STUDENT MARKS MATRIX ANALYSIS

Project Report — Python Programming

Project topic	Analyse student marks across subjects using a two-dimensional matrix
Row-wise statistics	Each student's total, average, highest and lowest subject mark
Column-wise statistics	Each subject's average, highest and lowest mark
Performance detection	Identify top performers and subjects below a set average threshold
Implementation	Python 3, console-based

Sample output below is an illustrative console transcript, not a captured screenshot. Run the program and capture your own screenshots if required by your instructor.

1. Problem Statement

Educational institutions maintain marks for many students across multiple subjects. Analysing these marks manually can take time and may lead to calculation errors. This project stores marks in a two-dimensional matrix and computes student-wise (row-wise) and subject-wise (column-wise) statistics. It also identifies the top-performing student or students and flags subjects whose average marks are below a user-defined threshold.

2. Objectives

• Store student names, subject names, and marks in a matrix.

• Calculate each student's total and average marks.

• Find each student's highest and lowest marks.

• Calculate each subject's average, highest, and lowest marks.

• Identify all students tied for the highest average.

• Detect subjects whose average is below a specified threshold.

• Display results in a clear, readable report.

3. Assumptions

• At least one student and one subject are entered.

• Student names and subject names are non-empty and unique (case-insensitive).

• Marks are whole numbers from 0 to 100, inclusive.

• Every student has one mark for every subject; missing marks are not allowed.

• The low-performance threshold is between 0 and 100 and is compared using average < threshold.

• A top performer is any student whose average equals the maximum student average.

4. Algorithm

1. Input the number of students and subjects.

2. Input unique student names and unique subject names.

3. For each student, input a mark for every subject and validate each mark is between 0 and 100.

4. Store the marks in a two-dimensional list (matrix), where each row represents one student.

5. For every row, calculate total, average, highest mark, and lowest mark.

6. For every column, calculate average, highest mark, and lowest mark.

7. Find the maximum student average and list all students with that average.

8. Ask for the low-performance threshold and flag subjects whose average is below it.

9. Print student-wise and subject-wise reports, top performers, and low-average subjects.

5. Flowchart

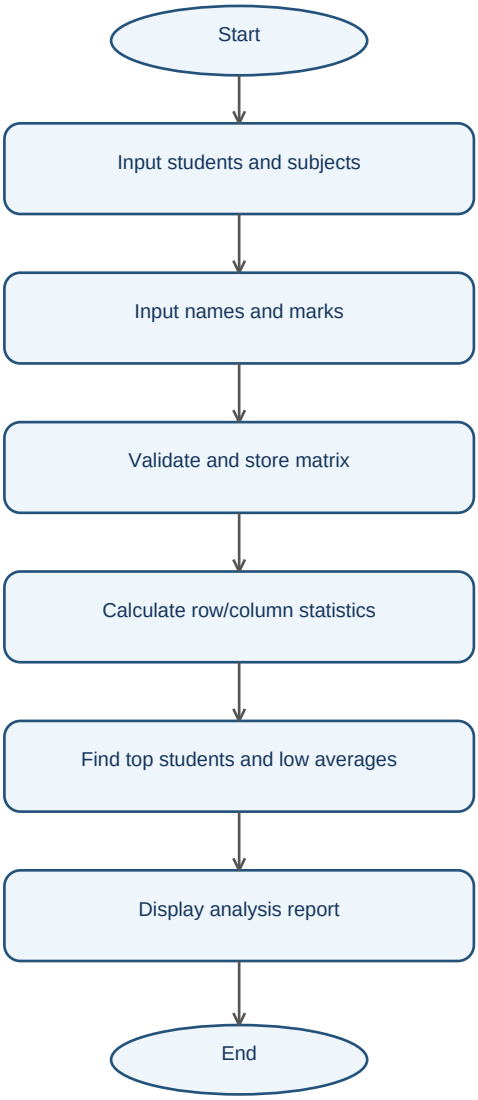

Figure 1. High-level flow of the student marks matrix analysis program.

6. Pseudocode

```
START
  INPUT student_count, subject_count
  INPUT unique student names
  INPUT unique subject names
  FOR each student
    FOR each subject
      INPUT mark
      WHILE mark is outside 0..100
        INPUT mark again
      END WHILE
      STORE mark in matrix
    END FOR
  END FOR

  FOR each student row
    CALCULATE total, average, maximum, minimum
  END FOR

  FOR each subject column
    CALCULATE average, maximum, minimum
  END FOR

  highest_average = maximum of student averages
  top_performers = all students with highest_average
  INPUT low_average_threshold
  low_subjects = subjects with average < threshold

  DISPLAY student statistics
  DISPLAY subject statistics
  DISPLAY top_performers and low_subjects
END
```

7. Python Source Code

```
def read_int(prompt, minimum, maximum):
    while True:
        try:
            value = int(input(prompt))
            if minimum <= value <= maximum:
                return value
            print(f"Enter a whole number from {minimum} to {maximum}.")
        except ValueError:
            print("Invalid input. Enter a whole number.")

def read_unique_names(count, label):
    names = []
    while len(names) < count:
        name = input(f"Enter {label} {len(names) + 1}: ").strip()
        if not name:
            print(f"{label.title()} cannot be empty.")
        elif name.casefold() in [n.casefold() for n in names]:
            print(f"{label.title()} must be unique.")
        else:
            names.append(name)
    return names

def main():
    student_count = read_int("Number of students: ", 1, 100)
    subject_count = read_int("Number of subjects: ", 1, 50)
    students = read_unique_names(student_count, "student name")
    subjects = read_unique_names(subject_count, "subject name")

    marks = []
    for student in students:
        row = []
        print(f"\nEnter marks for {student}:")
        for subject in subjects:
            row.append(read_int(f"  {subject} (0-100): ", 0, 100))
        marks.append(row)

    threshold = read_int("\nLow subject-average threshold (0-100): ", 0, 100)

    print("\nSTUDENT-WISE ANALYSIS")
    print(f"{'Student':<16}{'Total':>7}{'Average':>10}{'Highest':>9}{'Lowest':>8}")
    averages = []
    for name, row in zip(students, marks):
        total = sum(row)
        avg = total / len(row)
        averages.append(avg)
        print(f"{name:<16}{total:>7}{avg:>10.2f}{max(row):>9}{min(row):>8}")

    print("\nSUBJECT-WISE ANALYSIS")
```

```
    print(f"{'Subject':<16}{'Average':>10}{'Highest':>9}{'Lowest':>8}")
    subject_averages = []
    for col, subject in enumerate(subjects):
        values = [row[col] for row in marks]
        avg = sum(values) / len(values)
        subject_averages.append(avg)
        print(f"{subject:<16}{avg:>10.2f}{max(values):>9}{min(values):>8}")

    best_avg = max(averages)
    top_students = [students[i] for i, avg in enumerate(averages)
                    if abs(avg - best_avg) < 1e-9]
    low_subjects = [subjects[i] for i, avg in enumerate(subject_averages)
                    if avg < threshold]

    print("\nTOP PERFORMER(S): " + ", ".join(top_students))
    print(f"Highest average: {best_avg:.2f}")
    print(f"SUBJECTS WITH AVERAGE BELOW {threshold}:")
    if low_subjects:
        for subject in low_subjects:
            print(f"- {subject} (average {subject_averages[subjects.index(subject)]:.2f})")
    else:
        print("None")

if __name__ == "__main__":
    main()
```

8. Test Cases

Test	Scenario / sample data	Expected result
TC-01	Marks: Ana [80, 70], Bala [60, 90]; threshold 75	Ana avg 75.00; Bala avg 75.00; both top performers; subject averages 70 and 80; subject 1 flagged
TC-02	One student, one subject; mark 100; threshold 50	Average 100.00; student is top performer; no low-average subject
TC-03	Marks: A [40, 50], B [60, 70]; threshold 60	Student averages 45 and 65; subject averages 50 and 60; only first subject flagged
TC-04	Mark entered as -1 or 101	Input rejected; program asks for a mark from 0 to 100
TC-05	Duplicate student or subject name	Duplicate name rejected and requested again
TC-06	All students have the same average	All students tied at the maximum are listed as top performers

9. Sample Output / Console Screenshot

Sample input: 3 students (Anu, Ravi, Sara), 3 subjects (Maths, English, Science), marks [80, 70, 90], [60, 75, 65], [95, 85, 90], and threshold 75.

```
STUDENT-WISE ANALYSIS
Student           Total   Average  Highest  Lowest
Anu                 240     80.00       90      70
Ravi                200     66.67       75      60
Sara                270     90.00       95      85

SUBJECT-WISE ANALYSIS
Subject           Average  Highest  Lowest
Maths               78.33       95      60
English             76.67       85      70
Science             81.67       90      65

TOP PERFORMER(S): Sara
Highest average: 90.00
SUBJECTS WITH AVERAGE BELOW 75:
None
```

This is an illustrative console transcript rather than a captured screenshot. To create actual screenshots, run the program in IDLE or VS Code, enter the sample data, and capture the output window. On Windows, use Win + Shift + S to select the code or output area.

10. Results and Discussion

The program produces two complementary views of the same marks matrix. Student-wise analysis shows individual performance, while subject-wise analysis highlights subjects that may need additional teaching or practice. The threshold

is configurable, so an institution can choose its own definition of a low subject average. Ties are handled by reporting every student whose average equals the highest average.

11. Conclusion

The Student Marks Matrix Analysis project demonstrates the use of two-dimensional lists, nested loops, validation, aggregation, and formatted output in Python. It reduces manual calculation effort, identifies top-performing students, and helps detect subjects with lower average performance. Future enhancements could include reading marks from CSV or Excel, plotting charts, adding grades, and exporting the report.

12. How to Run

- Copy the source code into a file named student_marks_analysis.py.
- Run with Python 3 using: python student_marks_analysis.py
- Enter student and subject counts, names, marks, and the low-average threshold.
- Compare the result with the sample output and test cases.